

DAA Lab 7

Name: Sarthak Vashisht

Reg No: 24BRS1262

Topic: Backtracking – Sum of Subset, N-Queens and Graph Colouring

Aim:

To implement and analyze the Backtracking algorithms for the **Sum of Subset, N-Queens, and Graph Colouring** problems, obtain valid solutions for each problem, and evaluate their performance using execution time and the number of comparisons for different input sizes.

Theory:

Sum of Subset: The Sum of Subset problem involves finding a subset of given elements whose sum is equal to a specified target value. The Backtracking technique builds the solution incrementally by deciding whether each element should be included or excluded. If a partial solution cannot lead to the required sum, that path is abandoned and the algorithm goes back to explore another possibility. This systematic exploration allows the algorithm to find all possible subsets satisfying the given condition.

N-Queens: The N-Queens problem requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other. Queens cannot share the same row, column, or diagonal. Using Backtracking, queens are placed one row at a time. Whenever a queen placement results in a conflict with an already placed queen, that position is rejected and another position is tried. If no valid position exists in the current row, the algorithm backtracks to the previous row and changes the queen's position.

Graph Colouring: The Graph Colouring problem assigns colours to the vertices of a graph such that no two adjacent vertices have the same colour. Given a fixed number of available colours, the Backtracking method assigns colours to vertices one by one and checks whether each assignment is valid. If a colour creates a conflict with an adjacent vertex, another colour is attempted. When no available colour can be assigned, the algorithm backtracks and changes the colour of a previously processed vertex. This process continues until a valid colouring is obtained or all possibilities have been exhausted.

Algorithms:

1. Sum of Subset – Backtracking

Input:

- A set of n positive integers.
- A target sum S.

Output:

- All subsets whose sum is equal to S.

Algorithm

1. Start.
2. Read the number of elements n.
3. Read the elements of the set.
4. Read the target sum S.
5. Start with an empty subset and set the current sum to 0.

6. Consider the elements one by one.
7. For each element, make two choices:
 - Include the element in the current subset.
 - Exclude the element from the current subset.
8. If the current sum becomes equal to S, display the current subset as a valid solution.
9. If the current sum exceeds S, stop exploring that branch.
10. Recursively continue with the next element.
11. If a selected element does not lead to a solution, remove it from the subset and backtrack.
12. Continue until all possible choices have been explored.
13. Display all subsets having the required sum.
14. Stop.

2. N-Queens – Backtracking

Input:

- Number of queens N.

Output:

- A valid arrangement of N queens on an $N \times N$ chessboard.

Algorithm

1. Start.
2. Read the value of N.
3. Create an $N \times N$ chessboard and initialize all positions as empty.
4. Start placing queens from the first row.
5. For the current row, examine each column one by one.
6. Check whether placing a queen at the selected position is safe:
 - No queen should exist in the same column.
 - No queen should exist on the upper-left diagonal.
 - No queen should exist on the upper-right diagonal.
7. If the position is safe, place a queen there.
8. Recursively move to the next row.
9. If all N rows are successfully filled, a solution has been found.
10. If no safe position is available in the current row:
 - Remove the queen placed in the previous row.
 - Backtrack and try another position.
11. Continue until a valid arrangement is obtained or all possibilities have been checked.
12. Display the chessboard solution.
13. Stop.

3. Graph Colouring – Backtracking

Input:

- A graph containing n vertices.
- Number of available colours m.

Output:

- A valid assignment of colours to all vertices, if one exists.

Algorithm

1. Start.
2. Read the number of vertices n.

3. Read the adjacency matrix of the graph.
4. Read the number of available colours m .
5. Initialize the colour of every vertex to 0.
6. Start assigning colours from the first vertex.
7. For the current vertex, try each colour from 1 to m .
8. Check whether the selected colour is safe:
 - Examine all adjacent vertices.
 - The selected colour must not already be assigned to an adjacent vertex.
9. If the colour is safe, assign it to the current vertex.
10. Recursively move to the next vertex.
11. If all vertices receive valid colours, a solution has been found.
12. If no colour can be assigned to the current vertex:
 - Remove the colour assigned to the previous vertex.
 - Backtrack and try another colour.
13. Continue until a valid colouring is obtained or all possible colour assignments have been exhausted.
14. Display the colour assigned to each vertex.
15. Stop.

Code:**1) Sum of Subset:**

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace chrono;

long long comparisons = 0;
bool solutionFound = false;

void findSubset(vector<int>& arr, int n, int index,
               int currentSum, int target,
               vector<int>& currentSubset)
{
    comparisons++;

    // Target sum achieved
    if (currentSum == target)
    {
        solutionFound = true;

        cout << "Subset: { ";

        for (int x : currentSubset)
            cout << x << " ";

        cout << "}" << endl;

        return;
    }

    // No more elements
    if (index == n)
        return;
```

```

// If current sum exceeds target
comparisons++;

if (currentSum > target)
    return;

// Include current element
currentSubset.push_back(arr[index]);

findSubset(arr, n, index + 1,
           currentSum + arr[index],
           target,
           currentSubset);

currentSubset.pop_back();

// Exclude current element
findSubset(arr, n, index + 1,
           currentSum,
           target,
           currentSubset);
}

int main()
{
    srand(time(0));

    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n);

    // Generate random positive values
    for (int i = 0; i < n; i++)
    {
        arr[i] = rand() % 20 + 1;
    }
}

```

```

}

// Generate target sum
int target = 30;

vector<int> currentSubset;

comparisons = 0;
solutionFound = false;

auto start = high_resolution_clock::now();

findSubset(arr, n, 0, 0, target, currentSubset);

auto stop = high_resolution_clock::now();

auto duration =
    duration_cast<microseconds>(stop - start);

cout << "\n--- Performance Metrics ---" << endl;
cout << "Number of Elements : "
    << n << endl;

cout << "Target Sum      : "
    << target << endl;

cout << "Solution Found   : "
    << (solutionFound ? "Yes" : "No") << endl;

cout << "Execution Time    : "
    << duration.count()
    << " microseconds" << endl;

cout << "Total Comparisons : "
    << comparisons << endl;

return 0;
}

```

2)N-queens:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace chrono;

long long comparisons = 0;
bool solutionFound = false;

bool isSafe(vector<int>& board, int row, int col)
{
    for (int i = 0; i < row; i++)
    {
        comparisons++;

        // Same column
        if (board[i] == col)
            return false;

        comparisons++;

        // Same diagonal
        if (abs(board[i] - col) == abs(i - row))
            return false;
    }

    return true;
}

bool solveNQueens(vector<int>& board, int row, int n)
{
    comparisons++;

    if (row == n)
        return true;

    for (int col = 0; col < n; col++)
    {
        if (isSafe(board, row, col))
        {
            board[row] = col;

            if (solveNQueens(board, row + 1, n))
                return true;

            board[row] = -1;
        }
    }

    return false;
}

void displayBoard(vector<int>& board, int n)
{

```

```

cout << "\nChessboard Solution:\n";

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (board[i] == j)
            cout << "Q ";
        else
            cout << ". ";
    }

    cout << endl;
}

int main()
{
    srand(time(0));
    int n;

    cout << "Enter number of queens: ";
    cin >> n;

    vector<int> board(n, -1);

    comparisons = 0;

    auto start = high_resolution_clock::now();

    solutionFound = solveNQueens(board, 0, n);

    auto stop = high_resolution_clock::now();

    auto duration =
        duration_cast<microseconds>(stop - start);

    if (solutionFound)
        displayBoard(board, n);
    else
        cout << "\nNo solution exists." << endl;

    cout << "\n--- Performance Metrics ---" << endl;

    cout << "Number of Queens   : "
        << n << endl;

    cout << "Solution Found    : "
        << (solutionFound ? "Yes" : "No") << endl;

    cout << "Execution Time    : "
        << duration.count()
        << " microseconds" << endl;

    cout << "Total Comparisons : "
        << comparisons << endl;

    return 0;
}

```

3) Graph Colouring:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace chrono;

long long comparisons = 0;
bool solutionFound = false;

bool isSafe(int vertex, int colour,
            vector<vector<int>>& graph,
            vector<int>& colors,
            int n)
{
    for (int i = 0; i < n; i++)
    {
        comparisons++;

        if (graph[vertex][i] == 1 &&
            colors[i] == colour)
        {
            return false;
        }
    }

    return true;
}

bool graphColoring(int vertex,
                  int n,
                  int m,
                  vector<vector<int>>& graph,
                  vector<int>& colors)
{
    comparisons++;

    // All vertices coloured
    if (vertex == n)
        return true;

    // Try each colour
    for (int colour = 1; colour <= m; colour++)
    {
        if (isSafe(vertex, colour,
                    graph, colors, n))
        {
            colors[vertex] = colour;

            if (graphColoring(vertex + 1,
                              n, m,
                              graph, colors))
            {
                return true;
            }
        }
    }
}
```

```

        // Backtrack
        colors[vertex] = 0;
    }
}

return false;
}

int main()
{
    srand(time(0));

    int n;

    cout << "Enter number of vertices: ";
    cin >> n;

    int m = 3;

    vector<vector<int>>> graph(
        n,
        vector<int>(n, 0)
    );

    // Generate random undirected graph
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            int edge = rand() % 2;

            graph[i][j] = edge;
            graph[j][i] = edge;
        }
    }

    vector<int> colors(n, 0);

    comparisons = 0;

    auto start = high_resolution_clock::now();

    solutionFound =
        graphColoring(
            0,
            n,
            m,
            graph,
            colors
        );

    auto stop = high_resolution_clock::now();

    auto duration =
        duration_cast<microseconds>(stop - start);

    cout << "\n--- Performance Metrics ---" << endl;

```

```
cout << "Number of Vertices : "  
    << n << endl;  
  
cout << "Number of Colours : "  
    << m << endl;  
  
cout << "Solution Found   : "  
    << (solutionFound ? "Yes" : "No")  
    << endl;  
  
if (solutionFound)  
{  
    cout << "Colour Assignment : ";  
  
    for (int i = 0; i < n; i++)  
        cout << colors[i] << " ";  
  
    cout << endl;  
}  
  
cout << "Execution Time   : "  
    << duration.count()  
    << " microseconds" << endl;  
  
cout << "Total Comparisons : "  
    << comparisons << endl;  
  
return 0;  
}
```

Output:

1. Sum of Subset:

```
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 5
Subset: { 10 20 }

--- Performance Metrics ---
Number of Elements : 5
Target Sum         : 30
Solution Found     : Yes
Execution Time     : 593 microseconds
Total Comparisons  : 84
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 6
Subset: { 16 13 1 }

--- Performance Metrics ---
Number of Elements : 6
Target Sum         : 30
Solution Found     : Yes
Execution Time     : 557 microseconds
Total Comparisons  : 164
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 7

--- Performance Metrics ---
Number of Elements : 7
Target Sum         : 30
Solution Found     : No
Execution Time     : 7 microseconds
Total Comparisons  : 154
```

```

Total Comparisons : 134
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 8
Subset: { 6 1 20 3 }
Subset: { 18 9 3 }
Subset: { 18 9 3 }
Subset: { 1 9 20 }
Subset: { 1 9 20 }

--- Performance Metrics ---
Number of Elements : 8
Target Sum          : 30
Solution Found      : Yes
Execution Time       : 3620 microseconds
Total Comparisons    : 416
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 9
Subset: { 13 7 10 }
Subset: { 15 1 14 }
Subset: { 15 7 8 }
Subset: { 1 7 10 12 }
Subset: { 1 7 14 8 }
Subset: { 10 8 12 }
Subset: { 18 12 }

--- Performance Metrics ---
Number of Elements : 9
Target Sum          : 30
Solution Found      : Yes
Execution Time       : 3967 microseconds
Total Comparisons    : 586
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 10
Subset: { 6 13 8 3 }
Subset: { 6 14 7 3 }
Subset: { 6 14 10 }
Subset: { 6 17 7 }
Subset: { 6 13 8 3 }
Subset: { 13 14 3 }
Subset: { 13 17 }

```

```
PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./subset
Enter number of elements: 10
Subset: { 6 13 8 3 }
Subset: { 6 14 7 3 }
Subset: { 6 14 10 }
Subset: { 6 17 7 }
Subset: { 6 13 8 3 }
Subset: { 13 14 3 }
Subset: { 13 17 }
Subset: { 13 7 10 }
Subset: { 14 13 3 }
Subset: { 17 13 }
Subset: { 17 3 10 }
Subset: { 20 7 3 }
Subset: { 20 10 }
Subset: { 13 7 10 }

--- Performance Metrics ---
Number of Elements : 10
Target Sum         : 30
Solution Found      : Yes
Execution Time      : 7906 microseconds
Total Comparisons   : 587
PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> 
```

Number of Elements (n)	Execution Time (μs)	Number of Comparisons
5	593	84
6	557	164
7	7	154
8	3620	416
9	3967	586
10	7906	587

2. N-Queen:

```
• PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> g++ nqueen.cpp -o nqueen
• PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./nqueen
Enter number of queens: 5

Chessboard Solution:
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

--- Performance Metrics ---
Number of Queens : 5
Solution Found : Yes
Execution Time : 1 microseconds
Total Comparisons : 52
• PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./nqueen
Enter number of queens: 6

Chessboard Solution:
. Q . . . .
. . . Q . .
. . . . . Q
Q . . . . .
. . Q . . .
. . . . Q .

--- Performance Metrics ---
Number of Queens : 6
Solution Found : Yes
Execution Time : 11 microseconds
Total Comparisons : 673
```

```
PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./nqueen
Enter number of queens: 7
```

Chessboard Solution:

```
Q . . . . .
. . Q . . .
. . . . Q .
. . . . . Q
. Q . . . .
. . . Q . .
. . . . . Q
```

--- Performance Metrics ---

```
Number of Queens   : 7
Solution Found     : Yes
Execution Time      : 7 microseconds
Total Comparisons  : 185
```

```
PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./nqueen
Enter number of queens: 8
```

Chessboard Solution:

```
Q . . . . .
. . . . Q .
. . . . . Q
. . . . . Q
. . Q . . .
. . . . . Q
. Q . . . .
. . . Q . .
```

--- Performance Metrics ---

```
Number of Queens   : 8
Solution Found     : Yes
Execution Time      : 79 microseconds
Total Comparisons  : 4610
```

● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./nqueen

Enter number of queens: 9

Chessboard Solution:

```
Q . . . . . . . . .
. . Q . . . . . .
. . . . . Q . . .
. . . . . . Q . .
. Q . . . . . . .
. . . Q . . . . .
. . . . . . . Q
. . . . . Q . . .
. . . . Q . . . .
```

--- Performance Metrics ---

Number of Queens : 9
Solution Found : Yes
Execution Time : 34 microseconds
Total Comparisons : 1828

● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./nqueen

Enter number of queens: 10

Chessboard Solution:

```
Q . . . . . . . . .
. . Q . . . . . .
. . . . . Q . . .
. . . . . . Q . .
. . . . . . . Q
. . . . Q . . . .
. . . . . . . Q .
. Q . . . . . . .
. . . Q . . . . .
. . . . . Q . . .
```

--- Performance Metrics ---

Number of Queens : 10
Solution Found : Yes
Execution Time : 66 microseconds
Total Comparisons : 6042

Number of Queens (n)	Execution Time (μs)	Number of Comparisons
5	1	52
6	11	673
7	7	185
8	79	4610
9	34	1828
10	66	6042

3. Graph Colouring:

```
PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 5

--- Performance Metrics ---
Number of Vertices : 5
Number of Colours  : 3
Solution Found     : Yes
Colour Assignment  : 1 1 1 2 3
Execution Time     : 1 microseconds
Total Comparisons  : 37

PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 6

--- Performance Metrics ---
Number of Vertices : 6
Number of Colours  : 3
Solution Found     : Yes
Colour Assignment  : 1 2 2 2 3 1
Execution Time     : 16 microseconds
Total Comparisons  : 263

PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 7

--- Performance Metrics ---
Number of Vertices : 7
Number of Colours  : 3
Solution Found     : Yes
Colour Assignment  : 1 2 1 3 2 2 3
Execution Time     : 4 microseconds
Total Comparisons  : 112

PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 8

--- Performance Metrics ---
Number of Vertices : 8
Number of Colours  : 3
Solution Found     : No
Execution Time     : 53 microseconds
Total Comparisons  : 955
```

```
Total Comparisons : 112
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 8

--- Performance Metrics ---
Number of Vertices : 8
Number of Colours : 3
Solution Found : No
Execution Time : 53 microseconds
Total Comparisons : 955
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 9

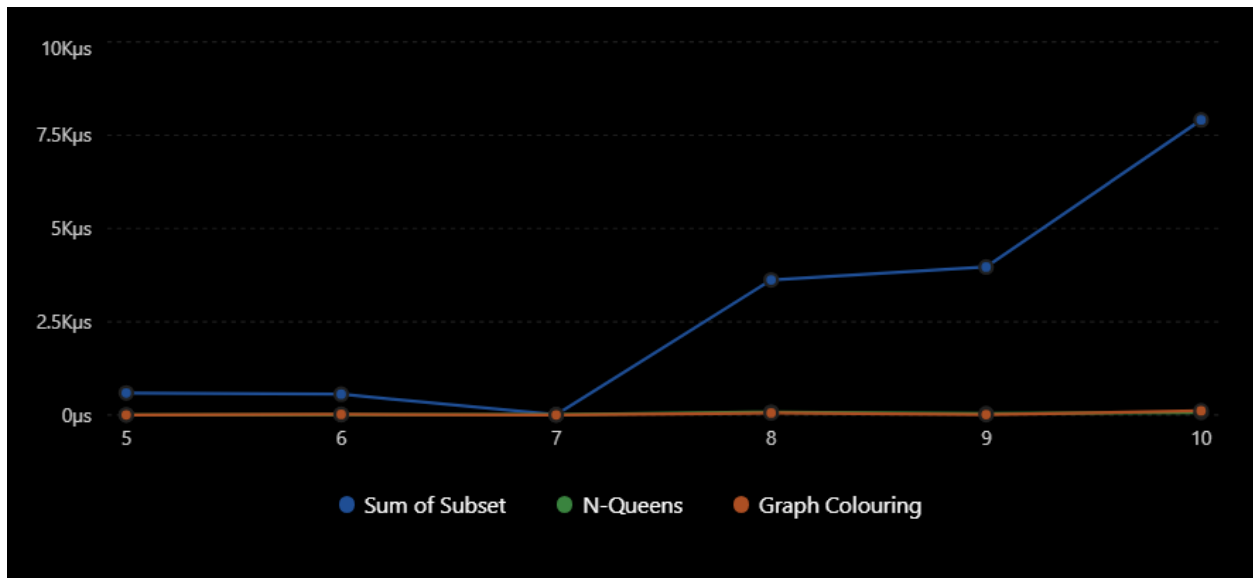
--- Performance Metrics ---
Number of Vertices : 9
Number of Colours : 3
Solution Found : Yes
Colour Assignment : 1 1 1 2 3 3 2 1 3
Execution Time : 11 microseconds
Total Comparisons : 180
● PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> ./graph
Enter number of vertices: 10

--- Performance Metrics ---
Number of Vertices : 10
Number of Colours : 3
Solution Found : No
Execution Time : 110 microseconds
Total Comparisons : 4087
○ PS C:\Users\Sarthak Vashisht\OneDrive\Desktop\24BRS1262> |
```

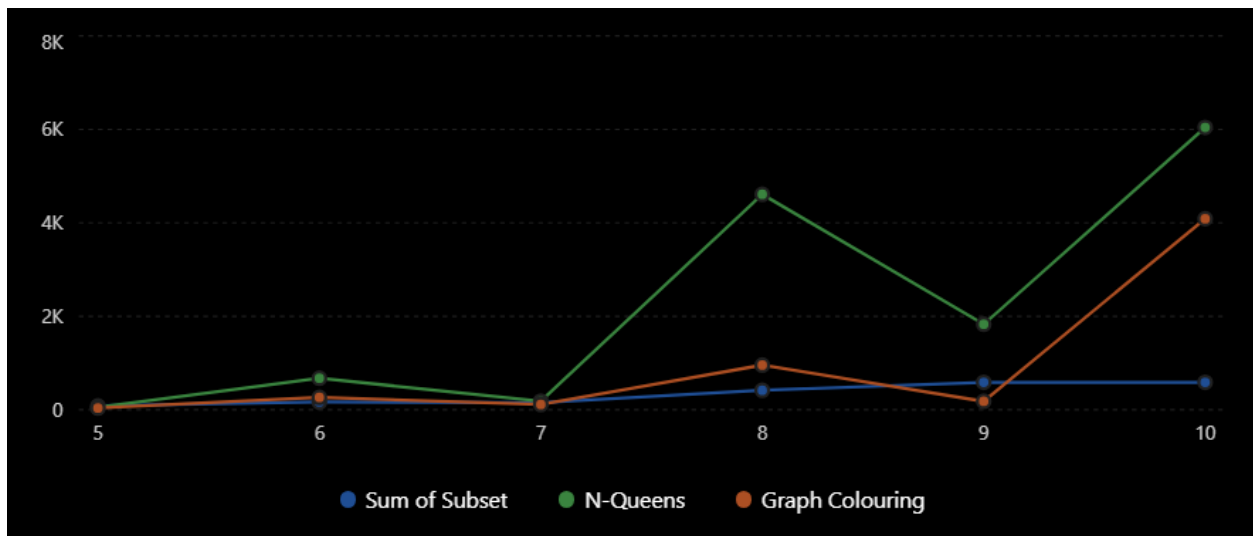
Number of Vertices (n)	Execution Time (μs)	Number of Comparisons
5	1	37
6	16	263
7	4	112
8	53	955
9	11	180
10	110	4087

Performance analysis:

1. Execution Time vs Number of Items



2. No of Comparisons vs Number of Items



Conclusion

The experiment successfully implemented three important Backtracking algorithms: Sum of Subset, N-Queens, and Graph Colouring. Their performance was tested using different input sizes. The results show that all three algorithms can find valid solutions by systematically exploring possible choices and backtracking whenever a selected path fails. As the input size increases, the computational requirements generally increase, demonstrating the significant search involved in Backtracking techniques.

Justification

Backtracking solves problems by constructing a solution step by step and abandoning a partial solution whenever it cannot lead to a valid answer. This makes it more efficient than blindly examining every complete possibility. The Sum of Subset algorithm explores different combinations of elements to achieve the target sum. N-Queens checks possible queen positions while eliminating arrangements that violate row, column, or diagonal constraints. Graph Colouring similarly tries different colour assignments and backtracks when adjacent vertices receive conflicting colours.

The experimental results support this behaviour. For example, N-Queens increased from 52 comparisons for $n = 5$ to 6,042 comparisons for $n = 10$, while Graph Colouring reached 4,087 comparisons at $n = 10$. Sum of Subset also showed an increase in computational work as the number of elements grew.

Observations

- **Sum of Subset:** The execution time generally increases as the number of elements increases, reaching 7,906 μs for $n = 10$ in the recorded runs.
- **N-Queens:** The number of comparisons rises significantly for larger boards, reaching 6,042 comparisons at $n = 10$.
- **Graph Colouring:** The algorithm shows considerable variation depending on whether a valid 3-colouring is found quickly or requires extensive backtracking. For $n = 10$, it required 4,087 comparisons and 110 μs .
- **Execution Time:** Small fluctuations are visible in the measured times because actual execution time depends on the system and the particular search path explored.
- **Backtracking Behaviour:** The performance is strongly influenced by the number of possible combinations and the amount of pruning performed during the search.

Result:

The Sum of Subset, N-Queens, and Graph Colouring problems were successfully implemented using the Backtracking technique. Their execution time and comparison counts were measured for different input sizes. The experiment demonstrates that Backtracking is effective for finding feasible solutions to combinatorial problems, but the computational effort can increase rapidly as the input size grows due to the large number of possible choices that may need to be explored.